

Лабораторная работа №7

*Выполнил: Беляев Дмитрий Михайлович
Студент 6203-010302D группы*

*Неужели финальная битва?
©Беляев Д.М.*

Ход выполнения

Первым делом было прочитано ТЗ. Я очень рад, что смог дойти до последней лабораторной за столь краткий срок(вообще, неделя – и все лабы сделать, это надо еще умудриться...)

Задание 1

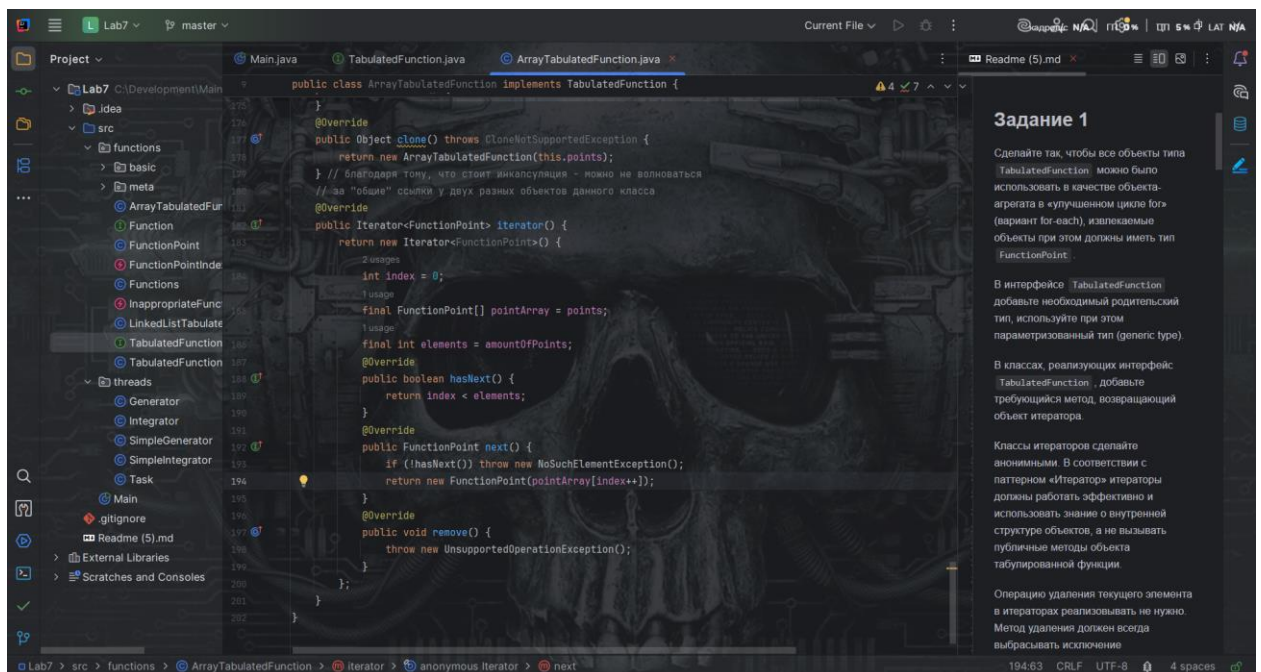
Добавим новый интерфейс(см скрин 1)

```
9 usages 2 implementations 2 related problems
public interface TabulatedFunction extends Function, Serializable, Cloneable, Iterable<FunctionPoint> {
```

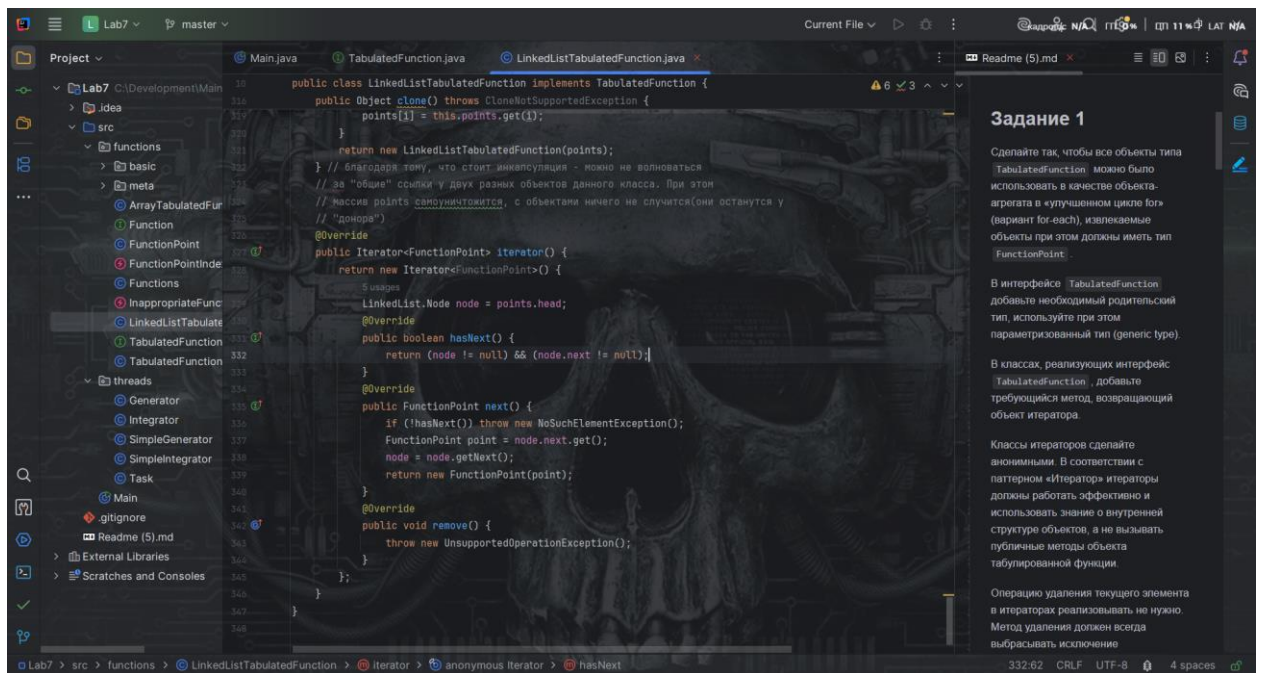
(скрин 1)

Понятно дело, появились проблемы(логично, т.к. нужно добавить новые методы)

Реализуем(см скрины 2-3)



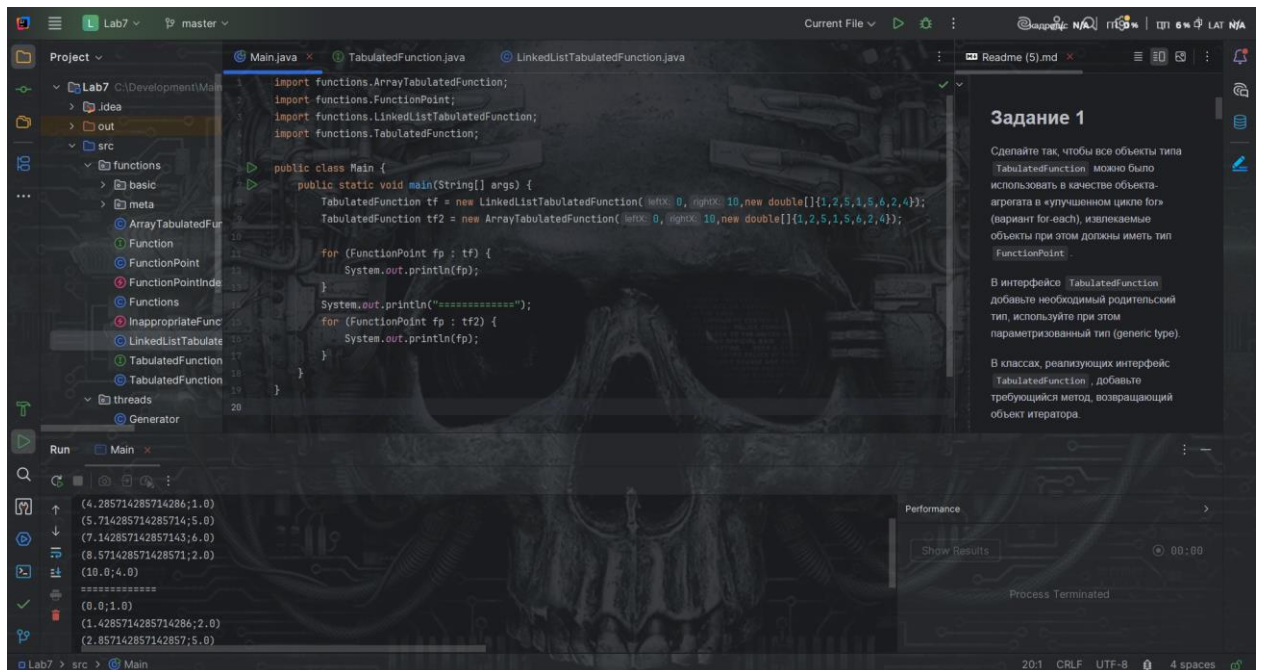
(скрин 2)



(скрин 3)

Реализация быстрая, в этом кстати и плюс не цикличного списка – понятно дело, где можно остановиться...

Теперь тестирование. В `Main` внесем нужные нам правки и запустим (см скрин 4)



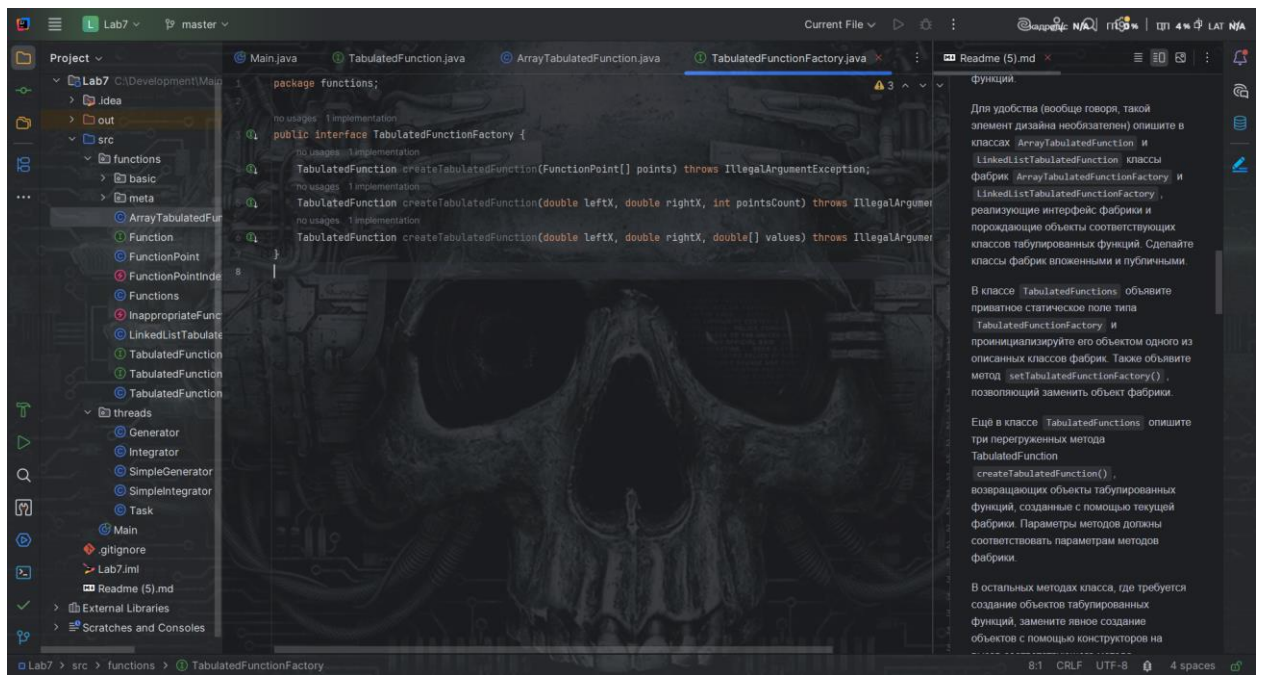
(скрин 4)

В принципе работает

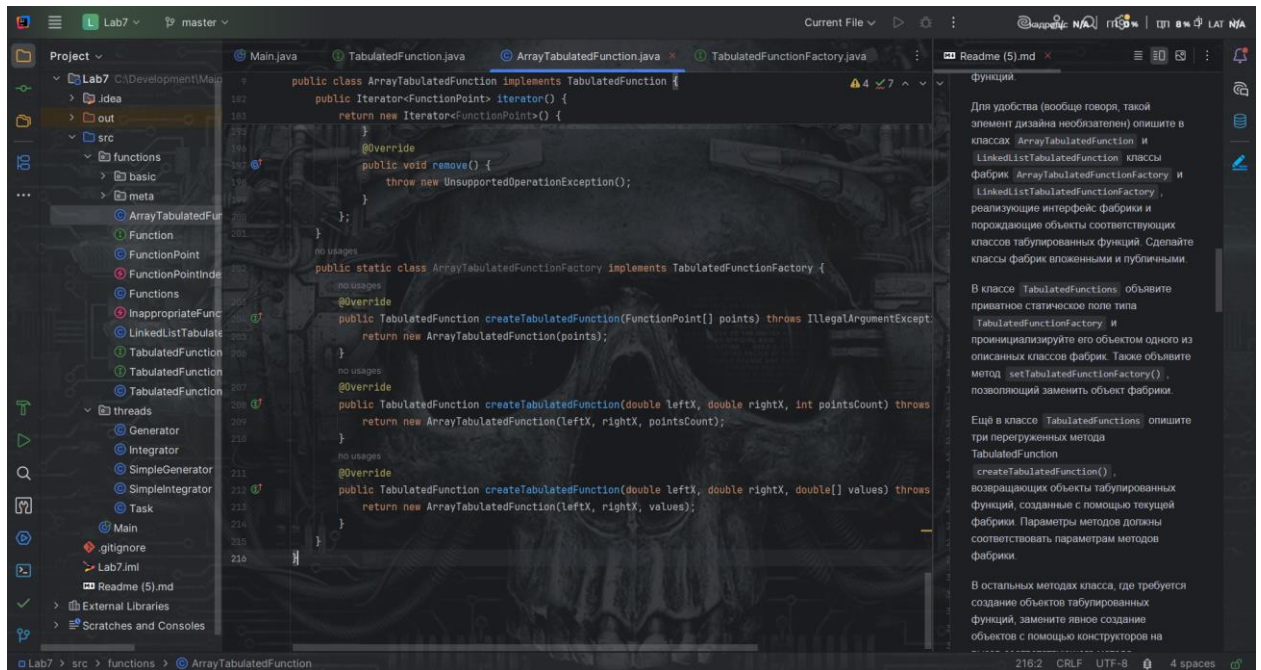
Задание выполнено

Задание 2

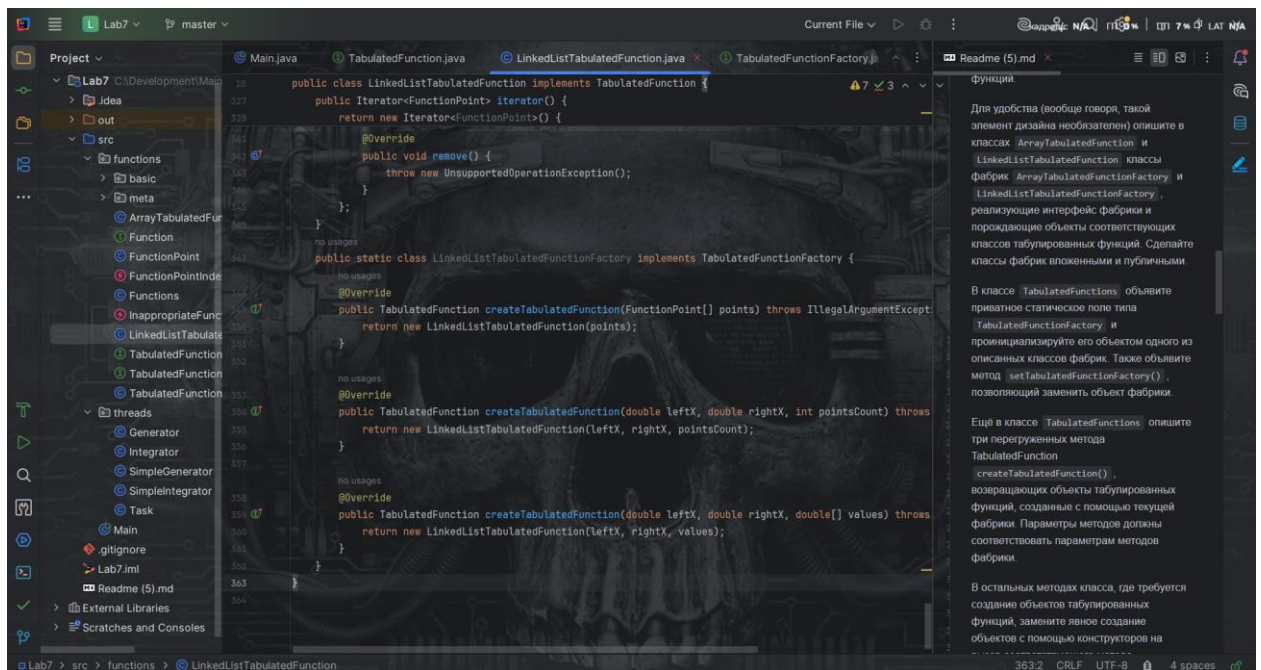
Сделаем интерфейс и рекомендации (см скрины 5-7)



(скрин 5)

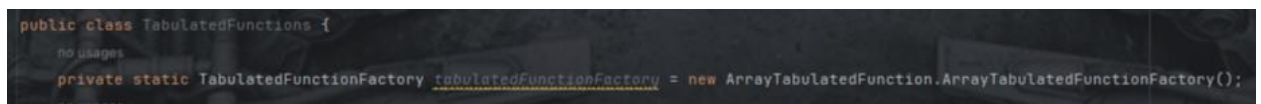


(скрин 6)

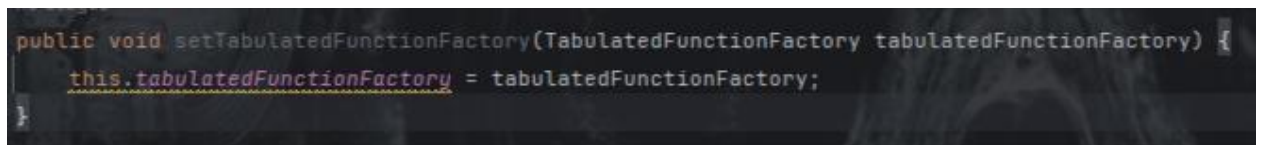


(скрин 7)

Реализуем поле и сеттер(см скрин 8-9)

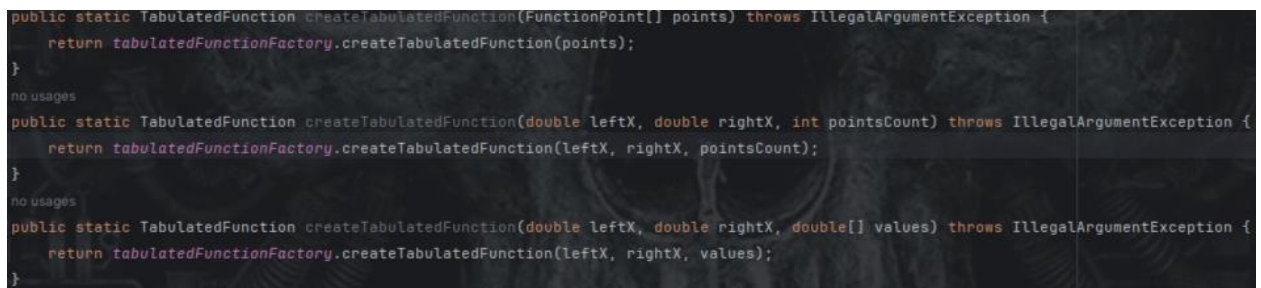


(скрин 8)



(скрин 9)

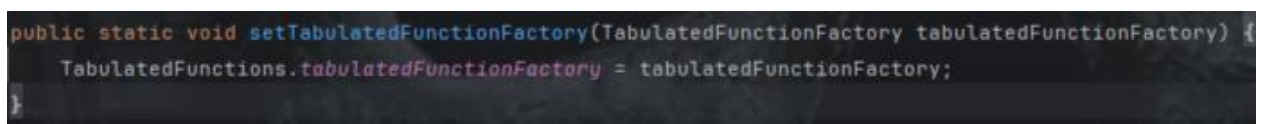
Честно, ваше дальнейшее задание я совершенно не понял. Нечего перегружать на самом деле, поэтому, см реализацию на скрине 10



(скрин 10)

Это не перегрузка, если так подумать, т.к. в TabulatedFunctions онли функции находятся

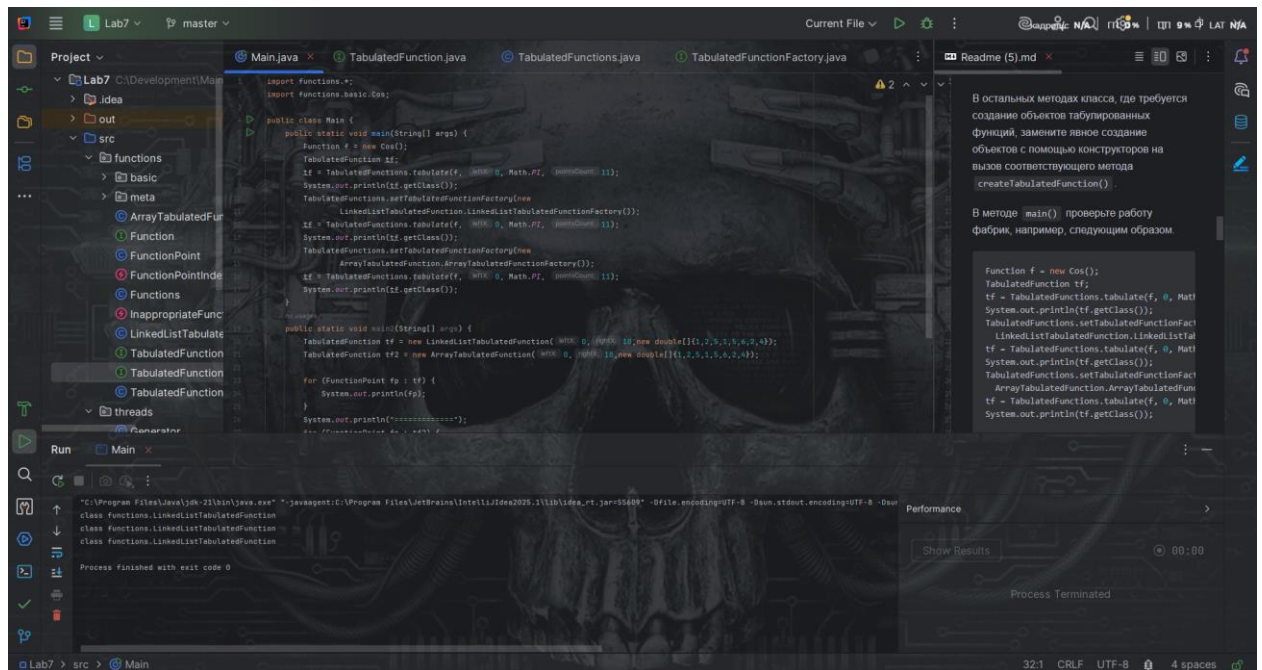
Еще в процессе обнаружил ошибку, исправил(см скрин 11)



(скрин 11)

Не было статик, да и поле должно быть получено через класс

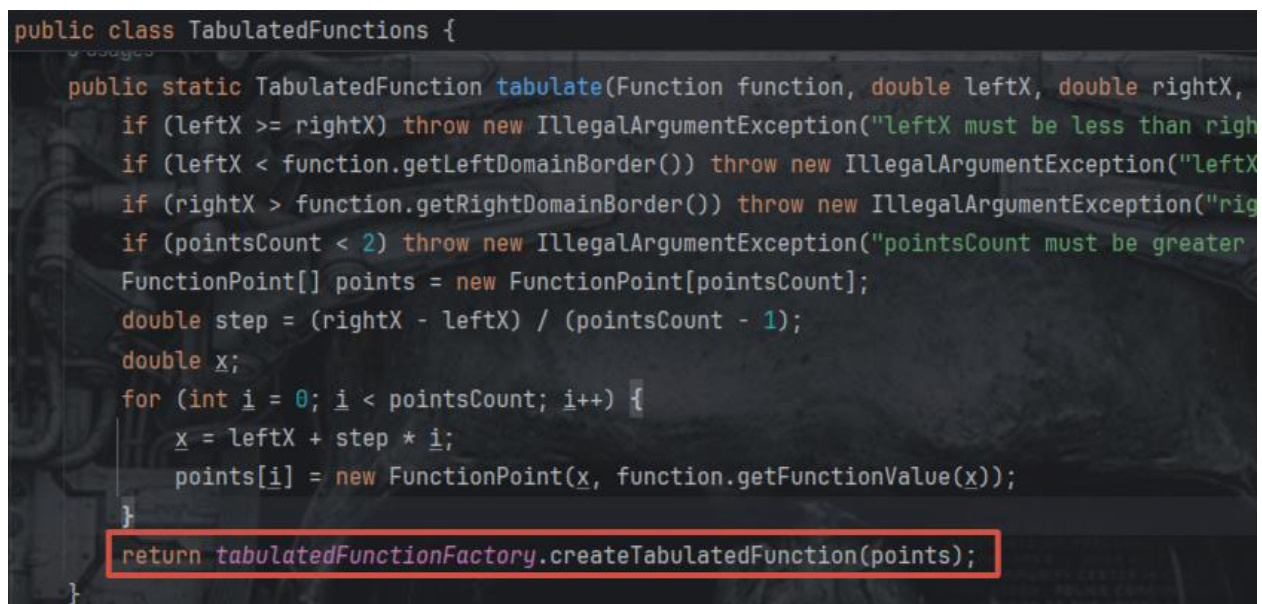
Взял ваш код, скопипастил, проверил(см скрин 12)



(скрин 12)

Похоже, я снова накосячил. Но нет. На самом деле, нужно самостоятельно изменить функции внутри данного класса...

Исправляем в общем...(см скрины 13-



(скрин 13)


```

public static TabulatedFunction inputTabulatedFunction(InputStream in) throws IOException {
    DataInputStream dis = new DataInputStream(in); // оборачиваем поток в нужный нам класс
    int amount = dis.readInt();
    FunctionPoint[] points = new FunctionPoint[amount];
    for (int i = 0; i < amount; i++) {
        points[i] = new FunctionPoint(dis.readDouble(), dis.readDouble());
    }
    return tabulatedFunctionFactory.createTabulatedFunction(points);
}

```

(скрин 14)

```

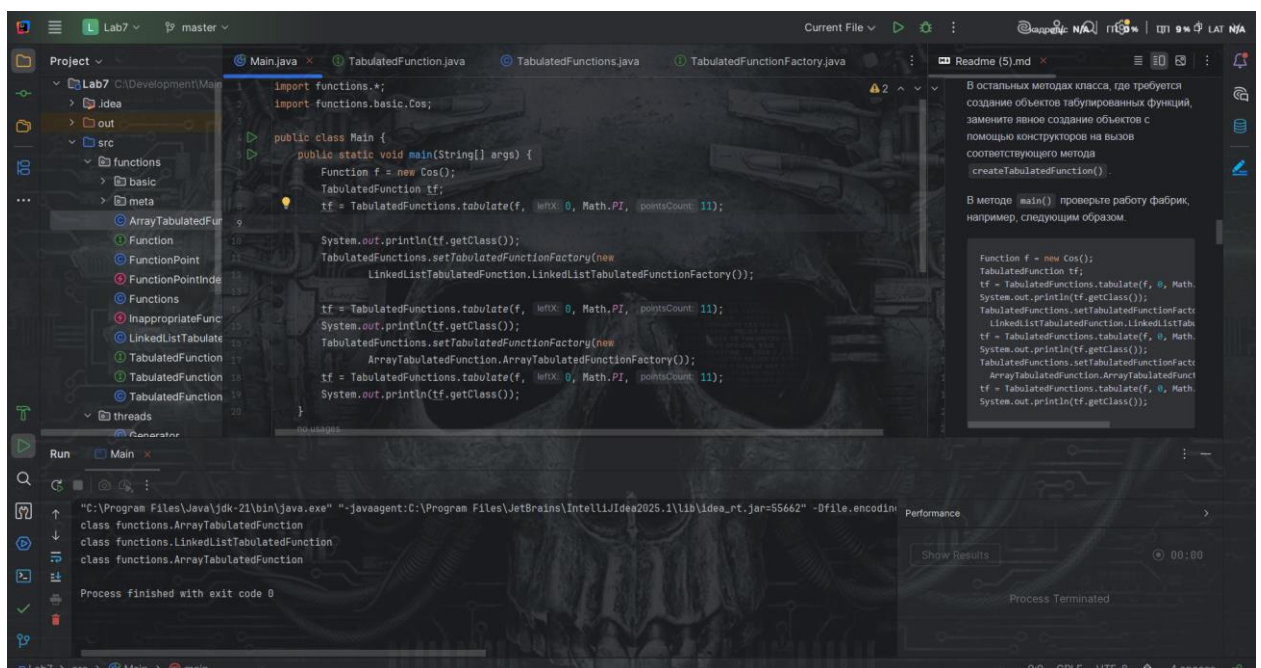
public static TabulatedFunction readTabulatedFunction(Reader in) throws IOException {
    // ...
    int amount = Integer.parseInt(st.sval);

    double x, y;
    FunctionPoint[] points = new FunctionPoint[amount];
    for (int i = 0; i < amount; i++) {
        st.nextToken();
        x = Double.parseDouble(st.sval);
        st.nextToken();
        y = Double.parseDouble(st.sval);
        points[i] = new FunctionPoint(x, y);
    }
    return tabulatedFunctionFactory.createTabulatedFunction(points);
}

```

(скрин 15)

Нигде не забыли, убрали везде реализацию через линкед. В общем все хорошо, запустим еще раз(см скрин 16)



(скрин 16)

Сначала я очень долго думал, а фиг ли ошибки нет... Потом вижу, ага, мы по умолчанию Array версию взяли... И все вопросы отпали

Задание выполнено

Задание 3

Вот и наступил момент истины.

Добавим новые 3 функции(см скрин 17)

```
public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> usedClass,  
    return usedClass.getDeclaredConstructor(points.getClass()).newInstance((Object) points);  
}  
no usages  
public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> usedClass,  
    return usedClass.getDeclaredConstructor(double.class, double.class, int.class).newInstance(leftX,  
}  
no usages  
public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> usedClass,  
    return usedClass.getDeclaredConstructor(double.class, double.class, double[].class).newInstance(Le  
}
```

(скрин 17)

И там кучу всевозможных ошибок может выкинуть... Но это все указываем, ответственность лежит на вызывающем.

И к сожалению я поздно очень заметил дополнительный пункт, так что добавим изменения(см скрин 18)

```
no usages  
public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> usedClass, Fu  
    try {  
        return usedClass.getDeclaredConstructor(points.getClass()).newInstance((Object) points);  
    } catch (Exception err) {  
        throw new IllegalArgumentException(err);  
    }  
}  
no usages  
public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> usedClass, do  
    try {  
        return usedClass.getDeclaredConstructor(double.class, double.class, int.class).newInstance(leftX  
    } catch (Exception err) {  
        throw new IllegalArgumentException(err);  
    }  
}  
no usages  
public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> usedClass, do  
    try {  
        return usedClass.getDeclaredConstructor(double.class, double.class, double[].class).newInstance(  
    } catch (Exception err) {  
        throw new IllegalArgumentException(err);  
    }  
}  
}
```

(скрин 18)

В общем, как и требовалось выполнить. Редактируем, добавляем новые версии функций(см скрины 19-21)

```
public static TabulatedFunction tabulate(Class<? extends TabulatedFunction> usedClass, Function function,
    if (leftX >= rightX) throw new IllegalArgumentException("leftX must be less than rightX");
    if (leftX < function.getLeftDomainBorder()) throw new IllegalArgumentException("leftX too low");
    if (rightX > function.getRightDomainBorder()) throw new IllegalArgumentException("rightX too high");
    if (pointsCount < 2) throw new IllegalArgumentException("pointsCount must be greater than 1");
    FunctionPoint[] points = new FunctionPoint[pointsCount];
    double step = (rightX - leftX) / (pointsCount - 1);
    double x;
    for (int i = 0; i < pointsCount; i++) {
        x = leftX + step * i;
        points[i] = new FunctionPoint(x, function.getFunctionValue(x));
    }
    try {
        return usedClass.getConstructor(FunctionPoint.class).newInstance((Object) points);
    } catch (Exception err) {
        throw new IllegalArgumentException(err);
    }
}
```

Argument is not assignable
FunctionPoint[] point

(скрин 19)

```
public static TabulatedFunction inputTabulatedFunction(Class<? extends TabulatedFunction> usedClass, Input
    DataInputStream dis = new DataInputStream(in); // оборачиваем поток в нужный нам класс
    int amount = dis.readInt();
    FunctionPoint[] points = new FunctionPoint[amount];
    for (int i = 0; i < amount; i++) {
        points[i] = new FunctionPoint(dis.readDouble(), dis.readDouble());
    }
    try {
        return usedClass.getDeclaredConstructor(points.getClass()).newInstance((Object) points);
    } catch (Exception err) {
        throw new IllegalArgumentException(err);
    }
}
```

(скрин 20)

```

public class TabulatedFunctions {
    public static TabulatedFunction readTabulatedFunction(Class<? extends TabulatedFunction> usedClass, R
        st.whitespaceChars( low: ' ', hi: ' ');

        st.wordChars( low: '0', hi: '9');
        st.wordChars( low: '-', hi: '-');
        st.wordChars( low: 'e', hi: 'e');
        st.wordChars( low: 'E', hi: 'E');
        st.wordChars( low: '.', hi: '.');

        st.nextToken();
        int amount = Integer.parseInt(st.sval);

        double x, y;
        FunctionPoint[] points = new FunctionPoint[amount];
        for (int i = 0; i < amount; i++) {
            st.nextToken();
            x = Double.parseDouble(st.sval);
            st.nextToken();
            y = Double.parseDouble(st.sval);
            points[i] = new FunctionPoint(x, y);
        }

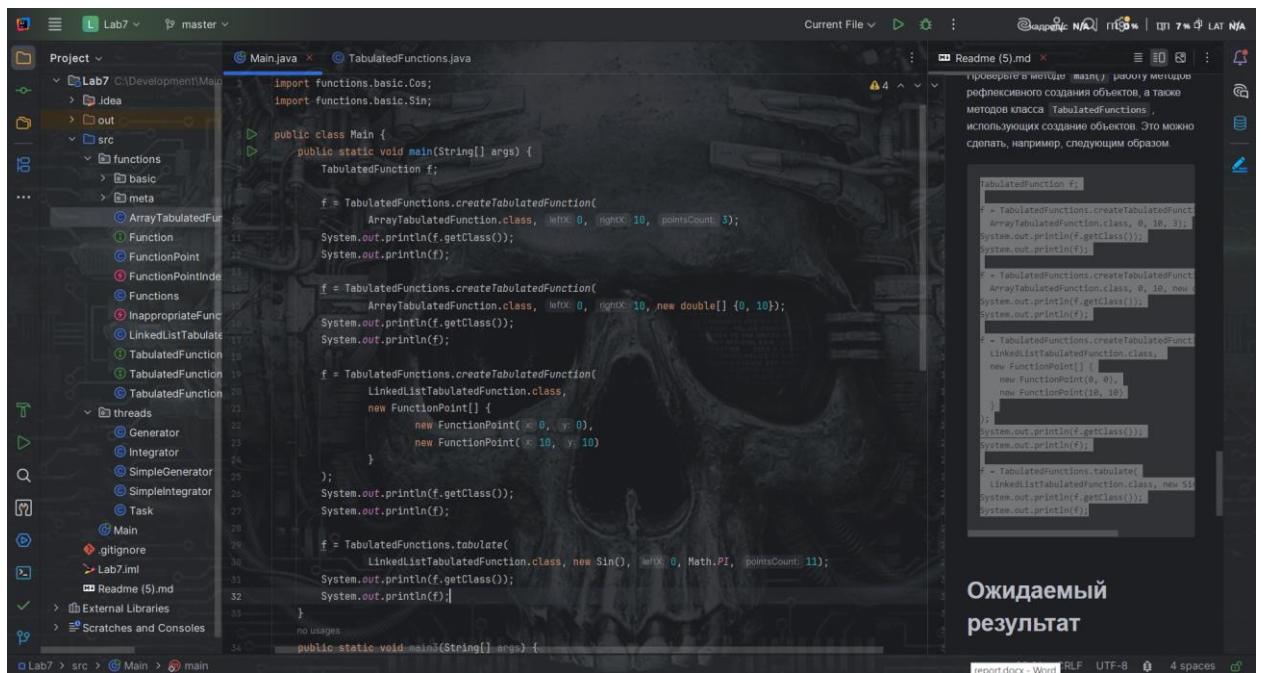
        try {
            return usedClass.getDeclaredConstructor(points.getClass()).newInstance((Object) points);
        } catch (Exception err) {
            throw new IllegalArgumentException(err);
        }
    }
}

```

(скрин 21)

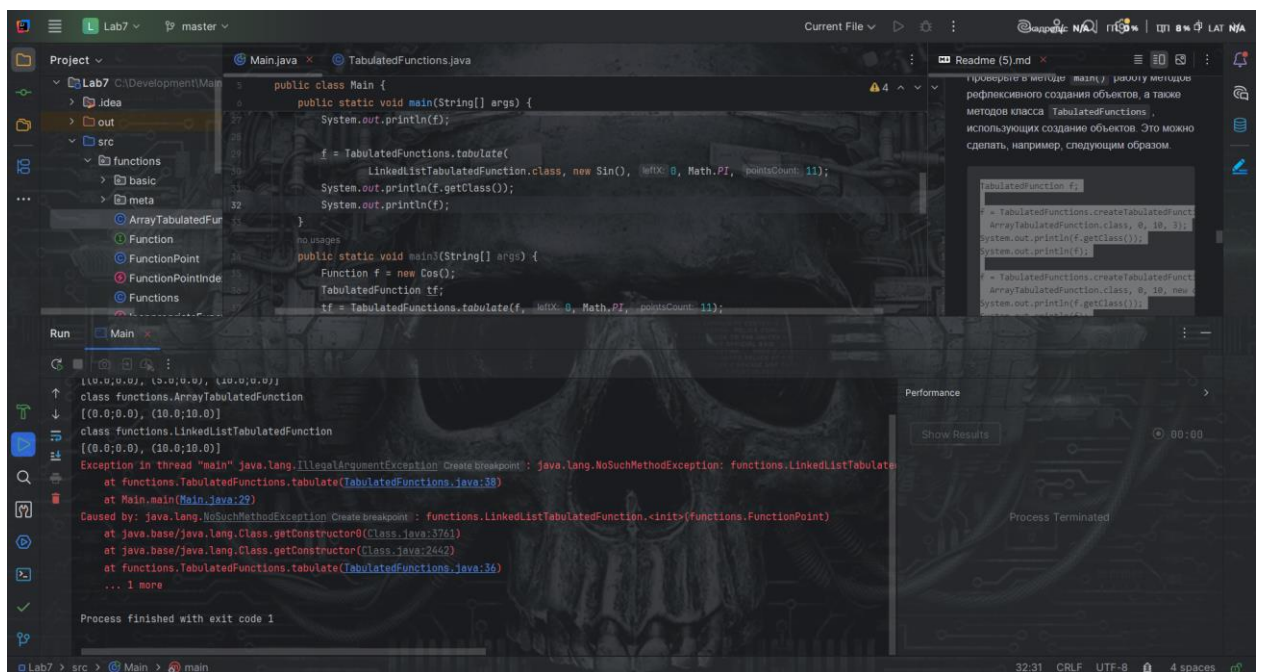
Остается протестировать нечестиво дело...

Буквально скопируем ваш код, спасим(см скрин 22)



(скрин 22)

Запустим. См скрин 23



(скрин 23)

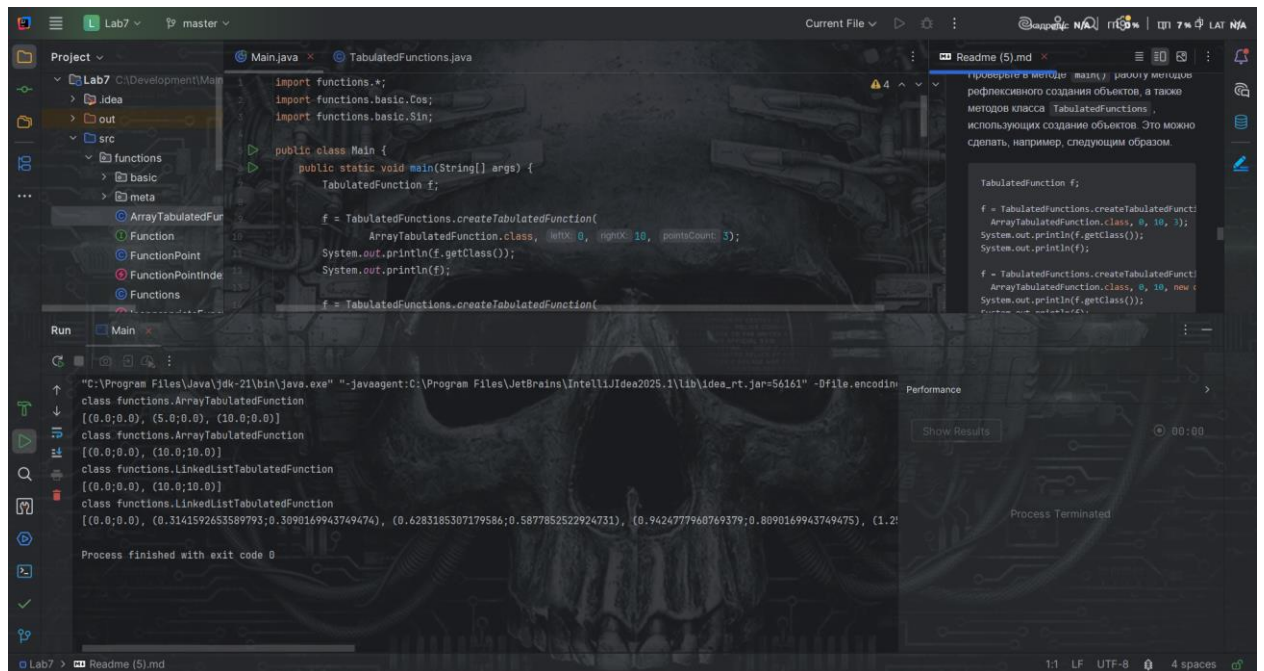
Исправим ошибку, сделанную по невнимательности...(см скрин 24) и запустим(см скрин 25)


```

public static TabulatedFunction tabulate(Class<? extends TabulatedFunction> usedClass, Function f) {
    x = leftX + step * i;
    points[i] = new FunctionPoint(x, function.getFunctionValue(x));
}
try {
    return usedClass.getConstructor(FunctionPoint.class).newInstance((Object) points);
} catch (Exception err) {
    throw new IllegalArgumentException(err);
}
}

```

(скрин 24)



(скрин 25)

Наконец то...

Лабораторная работа была выполнена(честно, я в шоке).